

Design Document

1. System Overview:

1.1 Host Computer

- The host computer is a general-purpose computer (i.e., your laptop or desktop) used to communicate with the Hardware Security Module (HSM) over a serial interface through a number of eCTF Tools.
- These tools will be used to initiate the various functionalities of the HSM device (see Functional Requirements) and to read back status messages and data. To simplify the system, all Host Tools communicating with the device will be written by the organizers.

1.2 Hardware Security Module (HSM)

- The main focus of your team's work is on the HSM device. An HSM is a generic security module that could be utilized as a sub component in a larger system.
- The primary function of this generic device is to store and transfer design files to other HSMs.
- The HSM will communicate to the Host Computer over the Management Interface. HSMs will communicate with each other over a separate Transfer Interface.

1.3 Files

- Secure file storage and transfer is the primary function of the HSM that your team will design.
- Files have a file name, an associated Permission Group, a Universally Unique Identifier (UUID), and contents. Files are stored in slots on the HSM.

1.4 Management Interface

- The management interface is a physical interface that is utilized by a user to instruct the HSM to perform certain actions. Most actions initiated from the management interface are protected by a PIN.
- This prevents unauthorized users from accessing sensitive files stored on the HSM. Communication on this interface will follow the protocol defined in the Functional Requirements.

1.5 Transfer Interface

- The transfer interface is physically distinct from the management interface.
- When prompted by the management interface, two HSMs will use the transfer interface to communicate files and file metadata between each other.

1.6 Permission Groups

- Permission groups are the critical security feature that HSMs rely on to authenticate one another. Every file belongs to one permission group. HSMs contain the permission data for groups based on the permissions they have for that group.
- For example, HSM A may have the read and receive permissions for the engineering group, but it does not have the write permission.
- At the same time, it could also have the write permission for the telemetry group, but neither the read nor the receive permission.

1.7 Receive Permission

- The receive permission enables HSMs to receive a file from a different HSM that contains files that

belong to the group.

- For example, if HSM A has the receive permission for the engineering data group and HSM B has files that belong to the engineering data group, HSM A will be able to request those files from HSM B. If A does not have the permission, B should refuse to transfer the file.

1.8 Write Permission

- The write permission enables an HSM to generate new files that belong to a specified permission group.
- For example, if HSM A has the write permission for the engineering group, it can create files in the engineering group.

1.9 Read Permission

- With the read permission, an HSM can return the file contents back to the user over the Management Interface.
- The mere fact that an HSM is storing a file does not mean that it should necessarily be able to return the file contents back to an authorized user. HSMs may contain files that they should not read.

2. Development Resources

Tools that are provided by ectf are:

- **3x un-keyed MSP-LITO-L2228 Boards (Design Phase Boards)**
 - These boards will be used for the development of your design. Instructions to run the host tools on a local computer to test the entire system using the physical hardware are found in eCTF Tools.
 - These devices will not be able to run Attack Phase designs provisioned by the eCTF organizers.
 - However, the development micro controllers can be used to practice attacks against designs in the Attack Phase that are compiled locally by the team from source.
- **3x keyed MSP-LITO-L2228 Boards (Attack Phase Boards)**
 - Boxes have small colored stickers to indicate that they contain provisioned attack boards.
 - These boards will be used for the Attack Phase to securely load other teams' designs that will be provided by the eCTF organizers.
 - These devices are configured for use in the Attack Phase and therefore will be unusable during the Design Phase.
- **4x XDS110-ETP-EVM USB Debuggers**
 - Used to program, communicate with, and debug the MSP-LITO-L2228 Board
- **20x 6" Female/Female Jumper Wires**
 - Used for connecting the debugger to the MSP-LITO-L2228 Board and for connecting the UART between two HSMs

2.1 MSP-LITO-L2228 Boards

The MSP-LITO-L2228 is a specialized development board designed for the 2026 MITRE Embedded Capture the Flag (eCTF) competition, featuring the low-power Texas Instruments MSPM0L2228 ARM Cortex-M0+ micro controller. It provides 256KB flash, 32KB SRAM, high-performance analog, and is used for designing secure bootloaders and firmware, with onboard LED/button interfaces for state management.

Key Features & Details

- Micro controller: TI MSPM0L2228 ARM® 32-bit Cortex-M0+ CPU, running at up to 32MHz.
- Purpose: Specifically tailored for the 2026 eCTF competition (secure bootloader development).
- I/O & Peripherals: Includes user LED (D1 - PB14), button (S2 - PB21) for update mode/resets, and various communication interfaces.
- Capabilities: Supports advanced analog features, AES encryption acceleration, and operates in low-power modes.
- Usage: Used in the eCTF to securely flash and run firmware.
- **Firmware Footprint:** The security logic, including the stored_pin_hash, is compiled into the 256KB Flash memory.
- **Volatile Operations:** Temporary hash buffers (like input_hash) are allocated within the 32KB SRAM to ensure sensitive data can be cleared after the verification cycle is complete

2.2 XDS110-ETP-EVM Debugger Board:

The XDS110-ETP-EVM is a compact, plug-and-play XDS110 debug probe designed by Texas Instruments for rapid debugging and flash programming. It supports UART and ARM SWD interfaces, designed mainly for MSPM0, MSP432, SimpleLink Wireless, and other TI micro controllers via USB 2.0. It acts as an easy-to-use alternative to on-board LaunchPad debuggers.

Key Features and Capabilities:

- Target Devices: Supports MSPM0, MSP432, CC26xx/13xx, CC32xx/31xx, Sitara, and C2000 MCUs.
- Interfaces: Features ARM SWD, UART, and supports JTAG/cJTAG.
- EnergyTrace: Includes EnergyTrace technology to measure target power consumption (up to 75mA natively, higher with expansion).
- Host Platforms: Compatible with Windows, Mac OS, and Linux.
- IDE Support: Fully integrated with TI Code Composer Studio (CCS), IAR, and Keil.

3. Protocols

3.1 UART Protocol:

- UART (Universal Asynchronous Receiver-Transmitter) is widely used in hardware-level asynchronous serial communication protocol that transfers data from devices using only two wires Transmit (TX) and Receive (RX) without any shared clock signal. It is to convert parallel data from a CPU into serial form, send it bit by bit, and reconvert it.
- It is used:
 - Management Interface (Host ↔ HSM) → UART0
 - Transfer Interface (HSM ↔ HSM) → UART1

3.2 Application-Level Protocol

Application-level protocols (it has Layer 7) are rules that define how the software applications (e.g., browsers, email clients) has to communicate over a network. They operate at the top of the TCP/IP stack to manage data exchange, such as HTTP/HTTPS for web traffic, SMTP/POP3 for email, and DNS for name resolution.

This is implemented in:

- host_messaging.c
- commands.c

3.3 Enhanced Security Protocol Layer

- **Constant-Time Verification:** The system implements a secure compare function to mitigate timing attacks.
- **Leakage Prevention:** By using the XOR (^) and OR-assignment (|=) operations across the entire 16-byte hash length, the execution time remains identical regardless of whether a PIN match or mismatch occurs.
- **Hash-Based Authentication:** The HSM does not store the user PIN in plaintext; instead, it stores a 128-bit (16-byte) MD5 hash of the authorized credential.
- **Input Sanitization:** Every authentication request includes a NULL pointer check to ensure system stability and prevent memory access violations during the verification process.

4. Cryptographic Implementation

- **Symmetric Encryption (AES-128):** The system implements AES-128 in a block-by-block direct mode (wc_AesEncryptDirect).
 - **Key Size:** All symmetric operations utilize a 16-byte (128-bit) key.
 - **Block Constraints:** Data must be a multiple of the 16-byte BLOCK_SIZE to be processed by the symmetric cipher.
- **Hashing (MD5):** The wc_Md5Hash function is used to produce a 16-byte digest for any arbitrary-length input data.
- **Error Handling:** Both encryption and decryption functions include validation checks to ensure input lengths are non-zero and properly aligned with the block size before processing begins.
- **Hashing Algorithm:** The system utilizes the MD5 algorithm via the hash() function to transform variable-length user input into a fixed-length digest for secure comparison.
- **Credential Storage:** A static, pre-computed hash (stored_pin_hash) is defined within the firmware to act as the ground truth for administrative access.
- **Algorithm Scope:** This cryptographic layer is integrated into security.c and serves as the primary barrier for the Management Interface.